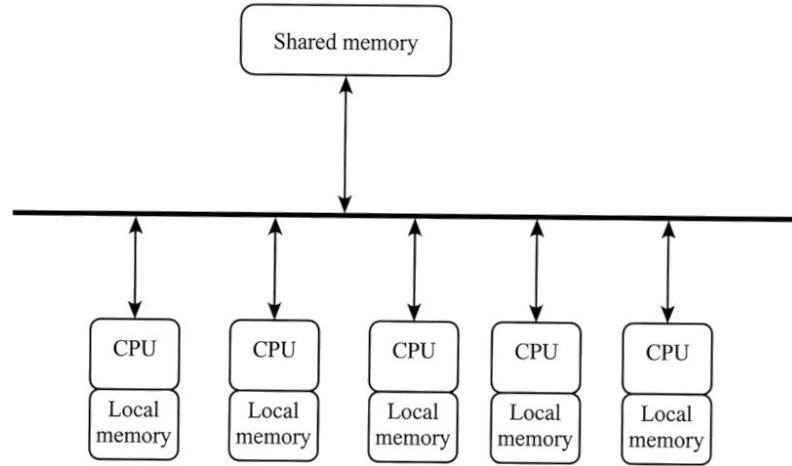


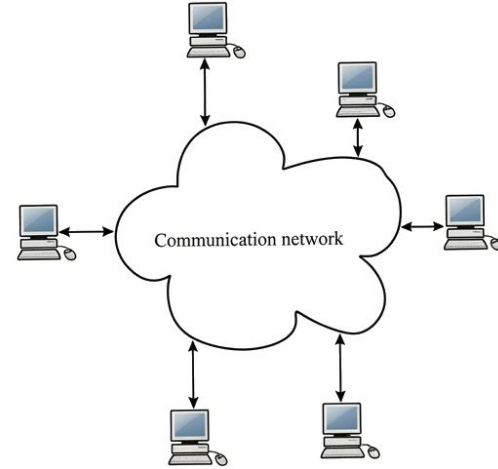
Distribuirani računarski sistemi

Distribuirano programiranje

Paralelni i distribuirani sistemi



Paralelni sistem



Distribuirani sistem

Prednosti distribuiranog računarskog sistema

- Skalabilnost
- Modularnost i heterogenost
- Dijeljenje i geografska distribucija podataka
- Dijeljenje resursa
- Pouzdanost
- Cijena

Zašto ne koristiti uvijek distribuirani računarski sistem?

- Zato što je brže da se ažurira dijeljena memorijska lokacija nego da se pošalje poruka preko mreže.

Komunikacija između procesa

- Soketi

- Komunikacija se zasniva na slanju sirovih bajtova podataka između IP adresa i portova.
- Programer mora definisati sopstveni protokol (format poruke), vršiti serijalizaciju (pretvaranje objekta u bajtove) i ručno upravljati konekcijom.
- Pružaju veću fleksibilnost, možeš komunicirati sa bilo kojim jezikom (npr. Java klijent priča sa C++ serverom).

- Remote Method Invocations (RMI)

- **specifično Java** rješenje omogućava da objekat na jednom računaru pozove metodu na objektu koji se nalazi na potpuno drugom računaru
- Programer ne vidi mrežne detalje (IP adrese, portove, stream-ove). Sve izgleda kao standardno objektno-orijentisano programiranje.

IP adrese

- Svaki računar u mreži mora imati jedinstvenu adresu koja služi za identifikaciju računara
- Hostname (npr. [google.com](https://www.google.com)) predstavlja je logičko ime koje ljudi lako čitaju i pamte.
- DNS je servis prevodi hostname u ip adresu.
- `Java.net.InetAddress` - ugrađena klasa za rad sa adresama.
 - `public byte[] getAddress()`
 - Returns the raw IP address of this `InetAddress` object.
 - `public static InetAddress getByName(String host)`
 - Determines the IP address of a host, given the host's name.
 - `public String.getHostAddress()`
 - Returns the IP address string "%d.%d.%d.%d".
 - `public String.getHostName()`
 - Returns the fully qualified host name for this address.
 - `public static InetAddress getLocalHost()`
 - Returns the local host.

Soketi TCP/UDP

UDP (User Datagram Protocol) – connectionless

- **Nema garancije:** Paketi mogu stići van reda, ili se mogu potpuno izgubiti u mreži.
- **Bez potvrde:** Ne postoji "handshaking" (rukovanje) niti potvrda prijema (acknowledgments).
- **Prednost:** Izuzetno je efikasan jer nema "overhead-a" (dodatnih podataka za provjeru grešaka).
- **Primjena:** Video streaming, online igre, VOIP – svuda gdje je bitnije da podatak stigne brzo nego da je svaki frejm savršen.

TCP (Transmission Control Protocol) – connection-oriented

- **Garantovana dostava:** Svaki paket mora biti potvrđen. Ako se izgubi, šalje se ponovo.
- **Redoslijed:** Paketi stižu tačno onim redom kojim su poslali.
- **Mana:** Manje je efikasan od UDP-a jer troši vrijeme i resurse na provjeru i uspostavljanje veze.
- **Primjena:** Web (HTTP), Email, prenos fajlova (FTP) – svuda gdje gubitak jednog bajta kvari cijelu poruku.

UDP Soketi

- Realizacija UDP komunikacije u Javi
- Individualno adresiranje - svaki paket se posebno adresira. Svaki paket nosi informaciju o tome kome ide (IP adresa i port).
- Dva paketa poslata jedan za drugim mogu putovati različitim putevima kroz internet.
- Zbog toga mogu stići bilo kojim redoslijedom (drugi može stići prije prvog).
- Prednost: Brzina
- `new DatagramSocket()` - sistem mu sam dodijeli slobodan port.
- `new DatagramSocket(1234)`
- `new DatagramSocket(int, InetAddress laddr)`

Metode klase DatagramSocket

- `close()`
 - Zatvara datagram soket i oslobađa resurse.
- `getLocalPort()`
 - Vraća broj porta na lokalnom hostu na koji je soket povezan.
- `getLocalAddress()`
 - Vraća lokalnu adresu (`InetAddress`) na koju je soket povezan.
- `receive(DatagramPacket p)`
 - Prima paket sa soketa.
 - Blokira izvršavanje dok paket ne stigne (osim ako je postavljen timeout).
 - Popunjava bafer paketa podacima, IP adresom i portom pošiljaoca.
- `send(DatagramPacket p)`
 - Šalje paket sa ovog soketa.
 - Paket mora sadržati podatke, dužinu, IP adresu i port odredišta.

Datagram paketi

- Da bi se podaci poslali preko DatagramSocket-a, oni moraju biti upakovani u objekte klase `java.net.DatagramPacket`.
- `DatagramPacket` sadrži:
 - Podaci (Payload): Niz bajtova (`byte[]`) koji se šalje ili prima.
 - Dužina (Length): Koliko je bajtova iz niza zapravo iskorišćeno za poruku.
 - Adresa (IP Address):
 - Pri slanju: Adresa primaoca.
 - Pri prijemu: Adresa pošiljaoca.
 - Port:
 - Pri slanju: Port na udaljenoj mašini.
 - Pri prijemu: Port sa kojeg je paket poslat.

TCP Soketi

- Connection-oriented: Mora se uspostaviti veza između pošiljaoca i primaoca pre slanja podataka.
- Garantovana isporuka: Protokol se brine da svaki bajt stigne na odredište.
- Očuvanje redosleda: Paketi se primaju tačno onim redosledom kojim su poslali.
- Error Recovery: Napredniji mehanizmi za oporavak od grešaka u prenosu.

TCP Soketi

- Klasa `java.net.Socket` se koristi za TCP komunikaciju u Javi na klijentskoj strani
- `public Socket(String host, int port)`
 - `host`: Ime servera (npr. `"localhost"` ili `"www.google.com"`) ili IP adresa.
 - `port`: Broj porta na kojem server sluša.
- Napomena: Ovaj konstruktor automatski pokušava da uspostavi vezu. Ako server nije dostupan, baca se `IOException`.

TCP Soketi

- Klasa `java.net.ServerSocket` se koristi za TCP komunikaciju u Javi na serverskoj strani
- Instanciranje: `ServerSocket server = new ServerSocket(9876);`
- Čekanje: Poziv `Socket klijent = server.accept();` (Server ovde stoji).
- Komunikacija: Koriste se `InputStream` i `OutputStream` dobijeni iz klijent soketa.
- Zatvaranje: Zatvaraju se tokovi i soketi nakon završetka posla.

Primjer: Linker

Napisati klasu Linker, koja omogućava komunikaciju između skupa procesa P_1 , $P_2 \dots P_n$.

Potrebno je da svaki proces može da šalje poruke svim ostalim procesima i da prima poruke od svih ostalih procesa

Java Remote Method Invocation (RMI)

- API koji omogućava da objekat na jednoj Java Virtual Machine (JVM) pozove metodu objekta na drugoj JVM, moguće i na drugom hostu
- Proces koji kreira poziv predstavlja klijenta
- Proces na kojem se nalazi objekat čija se metoda poziva zove se server
- U pozadini, naziv metoda i argumenti se upakuju u poruku koja se šalje serveru.
- U pozadini se odvija TCP komunikacija
- Server nakon izvršavanja metoda upakuje rezultat u poruku, i pošalje je
- Specifično za JVM, ne radi sa ostalim programskim jezicima
- Performanse, sporije od socketa

Terminologija

1. Stub (Klijentska strana)

- a. Stub je zastupnik (proxy) udaljenog objekta koji se nalazi na klijentu.
- b. Šta radi: Kada klijent pozove metodu, on zapravo poziva metodu na Stubu.
- c. Zadatak: Stub pakuje podatke (ovaj proces se zove Marshalling) i šalje ih preko mreže serveru. On "glumi" da je pravi objekat kako bi klijentski kod bio jednostavan.

2. Skeleton (Serverska strana)

- a. Skeleton je pomoćni objekat na strani servera.
- b. Šta radi: On prima mrežni zahtjev od Stuba.
- c. Zadatak: On raspakuje podatke (Unmarshalling), poziva pravu metodu na stvarnom objektu na serveru, uzima rezultat, ponovo ga pakuje i šalje nazad Stubu.

3. U novijim verzijama jave, ne vidite Skeleton fajlove, a Stub se generiše automatski u memoriji.

Remote objekat

Remote objekat je objekat koji se može pozvati iz druge JVM.

Objekat je opisan interfejsom. Interfejs mora da nasleđuje interfejs Remote.

Sve metode unutar interfejsa moraju da bacaju izuzetak tipa RemoteException.

Remote interface mora da bude dostupan i na klijentskoj i na serverskoj strani.

Na serverskoj strani treba implementirati i klasu koja implementira interfejs i nasljedjuje klasu `UnicastRemoteObject`

Java Remote Method Invocation (RMI)

- Prilikom pozivanja metoda
 - Argumenti se prenose kao poruke preko mreže
 - Povratna vrijednost se prenosi kao poruka preko mreže

Prenos objekata

- Primitivni tipovi se prenose po vrijednosti
- Lokalni objekti se šalju tako što se izvrši serijalizacija (pretvaranje objekta u niz bitova) i pošalje se serijalizovana verzija. Objekat treba da implementira interfejs Serializable.
- Ako objekat koji serijalizujemo sadrži reference na druge objekte, onda se i svi ti objekti moraju serijalizovati.
- Kao rezultat poziva metode preko RMI, dobija se lokalni objekat.
- Reference na objekte koji implementiraju Remote interfejs se predaju kao remote reference. (Stub za remote objekt se predaje)

Klijent

- Klijent treba da dobijer refernecu na Remote objekat
- Klasa `java.rmi.Naming` služi za uzimanje reference na remote objekat
- Referenca se dobija na osnovu Uniform Resource Locator (URL)-a
- URL je oblika:
 - `rmi://host:port/name`
 - `host` - host name na kojem se nalazi registry
 - `port` - port na kojem sluša registry
 - `name` - ime remote objekta

Thank you